

Specifica Sintattica e Semantica

Sistema di Gestione degli Accessi e delle Prenotazioni di un'Aula Studio

Traccia 2 — Prova Progetto PSD 2025/2026

Specifica delle operazioni sugli ADT del sistema: sintassi, semantica,
pre/post-condizioni, input/output ed effetti collaterali.

Indice

Indice	2
1. Introduzione e convenzioni di lettura	3
2. Panoramica degli ADT	4
2.1 Costanti di configurazione	4
3. ADT TabellaHashStudenti (Anagrafica)	5
3.1 Rappresentazione interna	5
3.2 Operazioni	5
4. ADT CodaAttesa (Lista d'attesa)	8
4.1 Rappresentazione interna	8
4.2 Operazioni	8
5. ADT TurnoAula (Aula e turno corrente)	11
5.1 Rappresentazione interna	11
5.2 Operazioni di prenotazione e accesso	11
5.3 Operazioni di gestione del turno e dell'orario.....	14
6. Operazioni di sistema, visualizzazione e persistenza	17
7. Programma principale (main.c).....	21
7.1 Variabili locali di main.....	21
8. Casi di Test (test.c).....	23
8.1 Variabili locali ricorrenti nei test.....	27

1. Introduzione e convenzioni di lettura

Questo documento contiene la specifica sintattica e semantica del Sistema di Gestione degli Accessi e delle Prenotazioni di un'Aula Studio (Traccia 2). La specifica e' organizzata per Tipo di Dato Astratto (ADT): per ciascun ADT vengono descritte la struttura logica e le operazioni che lo manipolano. Le funzioni di supporto e di interfaccia utente sono raccolte in una sezione finale.

Per ogni operazione viene fornita una scheda con i seguenti campi:

Sintassi: il prototipo della funzione, fedele all'implementazione nel codice.

Semantica: cosa fa l'operazione e le scelte progettuali rilevanti.

Parametri (input): significato di ciascun argomento.

Pre-condizione: cio' che deve valere prima della chiamata.

Post-condizione: cio' che e' garantito al termine.

Effetti collaterali: modifiche a strutture, file o stdout esterne al semplice valore di ritorno.

Output / Ritorna: il valore restituito e il suo significato.

2. Panoramica degli ADT

Il sistema si fonda su tre ADT principali, più alcuni tipi di valore di supporto.

ADT	Ruolo nel sistema
TabellaHashStudenti	Anagrafica degli studenti. Tabella hash con concatenazione (chaining); chiave = matricola. Garantisce ricerca e inserimento medi in $O(1)$.
TurnoAula	Stato dell'aula nella fascia corrente: array di posti, contatori e statistiche del turno e per fascia.
CodaAttesa	Lista d'attesa FIFO. Gestisce sia chi non trova posto sia le prenotazioni anticipate per turni futuri.

Tipi di valore (non opachi) e tipi enumerativi di supporto:

Tipo	Significato
OrarioVirtuale	Istante della giornata simulata (ora/minuti/secondi). Passato per valore.
FasciaOraria	MATTINA(0) < POMERIGGIO(1) < SERA(2). L'ordine è usato nei confronti.
StatoPosto	LIBERO, PRENOTATO, OCCUPATO. Ciclo: LIBERO -> PRENOTATO -> OCCUPATO -> LIBERO.

2.1 Costanti di configurazione

Costante	Significato
BUCKETS = 101	Numero di bucket della tabella hash; primo, per ridurre le collisioni dell'hash djb2.
MAX_POSTI = 100	Capienza dell'aula: dimensione dell'array di posti.

3. ADT TabellaHashStudenti (Anagrafica)

Struttura dati che memorizza gli studenti registrati, indicizzati per matricola tramite una tabella hash con concatenazione. Le collisioni sono risolte con liste concatenate (NodoStudente) all'interno di ogni bucket.

3.1 Rappresentazione interna

Tipo / Campo	Significato
<code>TabellaHashStudenti.tabella[BUCKETS]</code>	Array di puntatori a liste di <code>NodoStudente</code> (i bucket).
<code>NodoStudente.dati</code>	Studente memorizzato nel nodo.
<code>NodoStudente.next</code>	Puntatore al nodo successivo nello stesso bucket.
<code>Studente.matricola[12]</code>	Chiave univoca: 11 caratteri + terminatore.
<code>Studente.nome[60]</code>	Nome e cognome.
<code>Studente.corso_di_laurea[60]</code>	Corso di laurea.

3.2 Operazioni

calcola_hash

Sintassi	<code>int calcola_hash(char* matricola);</code>
Semantica	Calcola l'indice del bucket per una matricola con l'algoritmo djb2 ($\text{hash} * 33 + c$, seed 5381), scelto per la buona distribuzione su stringhe corte.
Parametri (input)	<code>char* matricola</code> — Stringa terminata da '\0'; ammesso NULL.
Pre-condizione	Nessuna.
Post-condizione	Funzione pura: non modifica alcuno stato.
Effetti collaterali	Nessuno.
Output / Ritorna	Intero in $[0, \text{BUCKETS}-1]$; 0 se matricola e' NULL.

crea_studente

Sintassi	<code>Studente crea_studente(char* matricola, char* nome, char* corso);</code>
Semantica	Costruisce uno <code>Studente</code> copiando i campi in modo sicuro (<code>strncpy</code> + terminazione esplicita).

Parametri (input)	<code>char* matricola</code> — Matricola (non NULL). <code>char* nome</code> — Nome e cognome (non NULL). <code>char* corso</code> — Corso di laurea (non NULL).
Pre-condizione	I tre parametri sono non NULL e terminati.
Post-condizione	Restituisce uno <code>Studiante</code> con campi terminati da <code>'\0'</code> anche in caso di troncamento.
Effetti collaterali	Nessuno (opera su una copia locale).
Output / Ritorna	Lo <code>Studiante</code> costruito, per valore.

inserisci_studente

Sintassi	<code>void inserisci_studente(TabellaHashStudenti* t, Studiante s);</code>
Semantica	Inserisce uno studente in testa al bucket ($O(1)$). Se la matricola esiste già, rifiuta senza sovrascrivere.
Parametri (input)	<code>TabellaHashStudenti* t</code> — Tabella (non NULL). <code>Studiante s</code> — Studente da inserire (copiato nel nodo).
Pre-condizione	<code>t</code> inizializzata.
Post-condizione	Se nuovo, lo studente è presente; in caso di duplicato o malloc fallita la tabella è invariata.
Effetti collaterali	Alloca un <code>NodoStudiante</code> (malloc); stampa esito su stdout; su errore di memoria stampa su stderr.
Output / Ritorna	<code>void</code> .

registra_studente

Sintassi	<code>void registra_studente(TabellaHashStudenti* t, char* matricola, char* nome, char* corso);</code>
Semantica	API pubblica di registrazione (<code>Studiante</code> è opaco). Costruisce lo studente, lo inserisce e, solo se l'inserimento avviene, registra l'evento nello storico.
Parametri (input)	<code>TabellaHashStudenti* t</code> — Anagrafica (non NULL). <code>char* matricola</code> — Matricola (non NULL). <code>char* nome</code> — Nome (non NULL). <code>char* corso</code> — Corso (non NULL).

Pre-condizione	t inizializzata.
Post-condizione	Studente inserito se nuovo; i duplicati non alterano i dati esistenti.
Effetti collaterali	Alloca memoria (tramite <code>inserisci_studente</code>); scrive su <code>storico_accessi.txt</code> ; stampa su <code>stdout</code> .
Output / Ritorna	<code>void</code> .

cerca_studente

Sintassi	<code>Studente* cerca_studente(TabellaHashStudenti* t, char* matricola);</code>
Semantica	Cerca uno studente per matricola: calcola il bucket, scorre la lista e ritorna il primo match.
Parametri (input)	<code>TabellaHashStudenti* t</code> — Tabella (NULL ammesso). <code>char* matricola</code> — Matricola da cercare (NULL ammesso).
Pre-condizione	Nessuna.
Post-condizione	Nessuna modifica allo stato.
Effetti collaterali	Nessuno.
Output / Ritorna	Puntatore allo Studente (interno alla tabella) oppure NULL se assente.

get_nome_studente

Sintassi	<code>const char* get_nome_studente(Studente* s);</code>
Semantica	Getter del nome (Studente e' opaco). Difensivo su NULL.
Parametri (input)	<code>Studente* s</code> — Puntatore a Studente; NULL ammesso.
Pre-condizione	Nessuna.
Post-condizione	Nessuna modifica allo stato.
Effetti collaterali	Nessuno.
Output / Ritorna	Puntatore al nome (sola lettura); stringa vuota se s e' NULL.

4. ADT CodaAttesa (Lista d'attesa)

Coda FIFO degli studenti in attesa. Mantiene puntatori a testa e coda per inserimento ed estrazione in $O(1)$. Ogni nodo memorizza anche la fascia richiesta, poichè la coda può contenere studenti per turni diversi (prenotazioni anticipate).

4.1 Rappresentazione interna

Tipo / Campo	Significato
<code>CodaAttesa.head</code>	Testa della coda (estrazione).
<code>CodaAttesa.tail</code>	Coda della coda (inserimento).
<code>CodaAttesa.dimensione</code>	Numero di elementi in coda.
<code>NodoAttesa.matricola[12]</code>	Matricola dello studente in attesa.
<code>NodoAttesa.data[11]</code>	Data del turno atteso.
<code>NodoAttesa.fascia</code>	Fascia richiesta.
<code>NodoAttesa.next</code>	Nodo successivo in coda.

4.2 Operazioni

accoda_studente

Sintassi	<code>void accoda_studente(CodaAttesa* coda, char* matricola, char* data, FasciaOraria fascia);</code>
Semantica	Inserisce uno studente in fondo alla coda (tail-insert, $O(1)$).
Parametri (input)	<code>CodaAttesa* coda</code> — Coda (NULL ammesso: caso difensivo). <code>char* matricola</code> — Matricola (copiata nel nodo). <code>char* data</code> — Data del turno (copiata nel nodo). <code>FasciaOraria fascia</code> — Fascia per cui si attende.
Pre-condizione	Coda inizializzata.
Post-condizione	dimensione +1; in caso di malloc fallita la coda resta invariata.
Effetti collaterali	Alloca un <code>NodoAttesa</code> ; stampa la posizione in coda su stdout; su errore stampa su stderr.
Output / Ritorna	void.

estrai_studente

Sintassi	<code>NodoAttesa* estrai_studente(CodaAttesa* coda);</code>
-----------------	---

Semantica	Estrae (dequeue) il primo nodo, restituendolo scollegato dalla lista. La free() e' a carico del chiamante.
Parametri (input)	CodaAttesa* coda — Coda (NULL ammesso).
Pre-condizione	Nessuna.
Post-condizione	dimensione -1 (se non vuota); head/tail coerenti; nodo restituito con next == NULL.
Effetti collaterali	Modifica i puntatori della coda; non libera il nodo (responsabilita' del chiamante).
Output / Ritorna	Puntatore al nodo estratto, oppure NULL se vuota/NULL.

svuota_coda

Sintassi	<code>void svuota_coda(CodaAttesa* coda);</code>
Semantica	Libera tutti i nodi lasciando la coda vuota ma valida. Usata nei reset (es. tra i test).
Parametri (input)	CodaAttesa* coda — Coda (NULL ammesso).
Pre-condizione	Nessuna.
Post-condizione	Coda vuota e coerente (head/tail NULL, dimensione 0).
Effetti collaterali	Libera memoria (free di tutti i nodi).
Output / Ritorna	void.

get_dimensione_coda

Sintassi	<code>int get_dimensione_coda(CodaAttesa* coda);</code>
Semantica	Getter del numero di studenti in coda (information hiding).
Parametri (input)	CodaAttesa* coda — Coda; NULL ammesso.
Pre-condizione	Nessuna.
Post-condizione	Nessuna modifica.
Effetti collaterali	Nessuno.

Output / Ritorna

Dimensione della coda; 0 se coda e' NULL.

5. ADT TurnoAula (Aula e turno corrente)

Rappresenta lo stato dell'aula nella fascia corrente: l'array dei posti, i contatori statistici del turno e quelli cumulativi per fascia. E' l'ADT centrale su cui agiscono prenotazioni, accessi e report.

5.1 Rappresentazione interna

Campo	Significato
<code>data[11]</code>	Data del turno (GG/MM/AAAA).
<code>fascia</code>	Fascia oraria corrente.
<code>posti[MAX_POSTI]</code>	Array dei posti fisici (Posto).
<code>posti_occupati</code>	Posti impegnati (PRENOTATO + OCCUPATO).
<code>totale_prenotazioni</code>	Prenotazioni del turno corrente.
<code>totale_checkin</code>	Check-in del turno corrente.
<code>totale_checkout</code>	Uscite del turno corrente.
<code>totale_no_show</code>	Prenotati non presentatisi a fine turno.
<code>totale_espulsi_da_coda</code>	Studenti rimasti in coda al cambio fascia.
<code>accessi_per_fascia[3]</code>	Accessi cumulativi: [0] MATTINA, [1] POMERIGGIO, [2] SERA.
<code>Posto.numero_posto</code>	Numero del posto (1..MAX_POSTI).
<code>Posto.stato</code>	LIBERO / PRENOTATO / OCCUPATO.
<code>Posto.matricola_studente[12]</code>	Assegnatario (valido se stato != LIBERO).
<code>Posto.ora_prenotazione</code>	Istante della prenotazione.

5.2 Operazioni di prenotazione e accesso

effettua_prenotazione

Sintassi	<pre>void effettua_prenotazione(TabellaHashStudenti* anagrafica, TurnoAula* aula, CodaAttesa* coda, char* matricola, FasciaOraria fascia_scelta, OrarioVirtuale ora_attuale);</pre>
Semantica	Prenota un posto. Se la fascia scelta e' quella corrente assegna un posto libero o, se pieno, accoda. Se la fascia e' futura accoda sempre (prenotazione anticipata). Se la fascia e' passata, rifiuta. Previene prenotazioni di matricole non registrate e doppie prenotazioni (sia su posto sia in coda).

Parametri (input)	TabellaHashStudenti* anagrafica — Anagrafica (non NULL). TurnoAula* aula — Stato aula (non NULL). CodaAttesa* coda — Coda (non NULL). char* matricola — Matricola dello studente. FasciaOraria fascia_scelta — Fascia per cui si prenota. OrarioVirtuale ora_attuale — Timestamp virtuale.
Pre-condizione	Sistema inizializzato; lo studente puo' essere o meno registrato (in caso negativo viene segnalato).
Post-condizione	Posto assegnato, o studente accodato, o nessuna modifica (errore/duplicato).
Effetti collaterali	Aggiorna posti e contatori; puo' accodare (malloc); scrive sullo storico; stampa su stdout.
Output / Ritorna	void.

effettua_checkin

Sintassi	<code>void effettua_checkin(TabellaHashStudenti* anagrafica, TurnoAula* aula, CodaAttesa* coda, char* matricola, OrarioVirtuale ora_attuale);</code>
Semantica	Registra l'ingresso fisico gestendo, in ordine: matricola non registrata; posto già assegnato (conferma/avviso); promozione dalla coda per la fascia corrente; attesa per fascia futura; ingresso diretto (posti liberi e coda vuota); inserimento in coda altrimenti.
Parametri (input)	<code>TabellaHashStudenti* anagrafica</code> — Anagrafica (non NULL). <code>TurnoAula* aula</code> — Stato aula (non NULL). <code>CodaAttesa* coda</code> — Coda (non NULL). <code>char* matricola</code> — Matricola dello studente. <code>OrarioVirtuale ora_attuale</code> — Timestamp virtuale.
Pre-condizione	Sistema inizializzato.
Post-condizione	Lo stato di aula/coda riflette lo scenario applicato; evento registrato.
Effetti collaterali	Aggiorna posti/contatori; può estrarre o inserire in coda (free/malloc); scrive su storico; stampa su stdout.
Output / Ritorna	<code>void</code> .

effettua_checkout

Sintassi	<code>void effettua_checkout(TabellaHashStudenti* anagrafica, TurnoAula* aula, CodaAttesa* coda, char* matricola, OrarioVirtuale ora_attuale);</code>
Semantica	Registra l'uscita. Liberato il posto, vi fa subentrare il primo studente in coda con fascia uguale alla corrente (stato PRENOTATO); se nessuno e' compatibile il posto torna LIBERO.
Parametri (input)	<code>TabellaHashStudenti* anagrafica</code> — Anagrafica (non NULL). <code>TurnoAula* aula</code> — Stato aula (non NULL). <code>CodaAttesa* coda</code> — Coda (non NULL). <code>char* matricola</code> — Matricola di chi esce. <code>OrarioVirtuale ora_attuale</code> — Timestamp virtuale.
Pre-condizione	Sistema inizializzato.
Post-condizione	Il posto torna LIBERO o passa al subentrante; storico aggiornato con CHECK-OUT e, se applicabile, SUBENTRO.
Effetti collaterali	Aggiorna posti/contatori; puo' liberare un nodo della coda (free); scrive su storico; stampa su stdout.
Output / Ritorna	void.

annulla_prenotazione

Sintassi	<code>void annulla_prenotazione(TabellaHashStudenti* anagrafica, TurnoAula* aula, CodaAttesa* coda, char* matricola, OrarioVirtuale ora_attuale);</code>
Semantica	Annulla una prenotazione. Scenario A: posto PRENOTATO -> liberato, con eventuale subentro dalla coda. Scenario B: prenotazione anticipata in coda -> rimossa, con decremento di totale_prenotazioni.
Parametri (input)	<code>TabellaHashStudenti* anagrafica</code> — Anagrafica (non NULL). <code>TurnoAula* aula</code> — Stato aula (non NULL). <code>CodaAttesa* coda</code> — Coda (non NULL). <code>char* matricola</code> — Matricola da annullare. <code>OrarioVirtuale ora_attuale</code> — Timestamp virtuale.
Pre-condizione	Sistema inizializzato.
Post-condizione	Vedi scenari A/B; se nulla da annullare, viene segnalato un errore.
Effetti collaterali	Aggiorna posti/contatori; puo' liberare un nodo (free); scrive su storico; stampa su stdout.

Output / Ritorna	void.
-------------------------	-------

5.3 Operazioni di gestione del turno e dell'orario

cambio_fascia_automatica

Sintassi	<code>void cambio_fascia_automatica(TurnoAula* aula, CodaAttesa* coda, FasciaOraria nuova_fascia);</code>
Semantica	Chiude il turno e prepara il successivo: conta no-show e checkout forzati; pulisce SELETTIVAMENTE la coda (rimuove solo i nodi della fascia conclusa, mantiene i futuri); resetta posti e contatori; aggiorna la fascia; promuove in FIFO i nodi della nuova fascia ai posti liberi.
Parametri (input)	<code>TurnoAula* aula</code> — Stato aula (non NULL). <code>CodaAttesa* coda</code> — Coda (non NULL). <code>FasciaOraria nuova_fascia</code> — Fascia da attivare.
Pre-condizione	Tipicamente <code>nuova_fascia</code> diversa dalla corrente.
Post-condizione	Aula azzerata e nella nuova fascia; coda epurata dei nodi conclusi; nodi compatibili promossi ai posti.
Effetti collaterali	Libera nodi della coda (free); aggiorna posti/contatori; scrive su storico; stampa su stdout.
Output / Ritorna	void.

aggiorna_orario_automatico

Sintassi	<code>void aggiorna_orario_automatico(OrarioVirtuale* ora, time_t* ultimo_aggiornamento, TurnoAula* aula, CodaAttesa* coda);</code>
Semantica	Avanza l'orario virtuale in base al tempo reale trascorso (1s reale = 120s virtuali) e innesca il cambio fascia al superamento delle soglie. L'avanzamento secondo-per-secondo e' robusto ai balzi dell'orologio.
Parametri (input)	<code>OrarioVirtuale* ora</code> — Orario virtuale, modificato in place. <code>time_t* ultimo_aggiornamento</code> — Timestamp reale, modificato in place. <code>TurnoAula* aula</code> — Stato aula (per il cambio fascia). <code>CodaAttesa* coda</code> — Coda (per il cambio fascia).
Pre-condizione	Tutti i puntatori non NULL.

Post-condizione	Orario avanzato; eventuale cambio fascia gestito.
Effetti collaterali	Modifica <code>*ora</code> e <code>*ultimo_aggiornamento</code> ; <code>puo'</code> invocare <code>cambio_fascia_automatica</code> (con i suoi effetti).
Output / Ritorna	<code>void</code> .

orario_in_secondi

Sintassi	<code>long orario_in_secondi(OrarioVirtuale o);</code>
Semantica	Converte un <code>OrarioVirtuale</code> in secondi dalla mezzanotte, per confronti numerici.
Parametri (input)	<code>OrarioVirtuale o</code> — Orario da convertire.
Pre-condizione	Nessuna.
Post-condizione	Funzione pura.
Effetti collaterali	Nessuno.
Output / Ritorna	Secondi dalla mezzanotte (<code>long</code>).

is_orario_valido

Sintassi	<code>int is_orario_valido(OrarioVirtuale adesso, FasciaOraria fascia);</code>
Semantica	Verifica se un orario ricade nella fascia (MATTINA 09-13, POMERIGGIO 14-18, SERA 18-22).
Parametri (input)	<code>OrarioVirtuale adesso</code> — Orario da verificare. <code>FasciaOraria fascia</code> — Fascia di riferimento.
Pre-condizione	Nessuna.
Post-condizione	Funzione pura.
Effetti collaterali	Nessuno.
Output / Ritorna	1 se l'orario ricade nella fascia, 0 altrimenti.

get_fascia_aula

Sintassi	<code>FasciaOraria get_fascia_aula(TurnoAula* aula);</code>
-----------------	---

Semantica	Getter della fascia corrente.
Parametri (input)	TurnoAula* aula — Stato aula; NULL ammesso.
Pre-condizione	Nessuna.
Post-condizione	Nessuna modifica.
Effetti collaterali	Nessuno.
Output / Ritorna	La fascia corrente; MATTINA se NULL.

get_posti_occupati_totali

Sintassi	int get_posti_occupati_totali(TurnoAula* aula);
Semantica	Getter dei posti impegnati (PRENOTATO + OCCUPATO).
Parametri (input)	TurnoAula* aula — Stato aula; NULL ammesso.
Pre-condizione	Nessuna.
Post-condizione	Nessuna modifica.
Effetti collaterali	Nessuno.
Output / Ritorna	Numero di posti occupati; 0 se NULL.

get_data_aula

Sintassi	char* get_data_aula(TurnoAula* aula);
Semantica	Getter della data del turno corrente.
Parametri (input)	TurnoAula* aula — Stato aula; NULL ammesso.
Pre-condizione	Nessuna.
Post-condizione	Nessuna modifica.
Effetti collaterali	Nessuno.
Output / Ritorna	Puntatore alla data; NULL se aula e' NULL.

6. Operazioni di sistema, visualizzazione e persistenza

inizializza_sistema

Sintassi	<code>void inizializza_sistema(TabellaHashStudenti* t, TurnoAula* aula, CodaAttesa* coda);</code>
Semantica	Inizializza strutture già allocate: azzerava bucket e contatori, svuota la coda, imposta i posti LIBERO, fissa la data reale e la fascia MATTINA.
Parametri (input)	<code>TabellaHashStudenti* t</code> — Tabella allocata. <code>TurnoAula* aula</code> — Aula allocata. <code>CodaAttesa* coda</code> — Coda allocata.
Pre-condizione	Le tre strutture sono allocate.
Post-condizione	Bucket NULL, coda vuota, posti LIBERO, contatori a 0, data e fascia di default impostate.
Effetti collaterali	Legge l'ora di sistema (time/localtime); stampa su stdout.
Output / Ritorna	void.

inizializza_sistema_dinamico

Sintassi	<code>void inizializza_sistema_dinamico(TabellaHashStudenti** t, TurnoAula** aula, CodaAttesa** coda);</code>
Semantica	Alloca dinamicamente le tre strutture e le inizializza. Usa doppi puntatori per modificare le variabili del chiamante; su malloc fallita termina con exit(1).
Parametri (input)	<code>TabellaHashStudenti** t</code> — Indirizzo del puntatore alla tabella. <code>TurnoAula** aula</code> — Indirizzo del puntatore all'aula. <code>CodaAttesa** coda</code> — Indirizzo del puntatore alla coda.
Pre-condizione	I tre parametri sono indirizzi di variabili valide.
Post-condizione	*t, *aula, *coda puntano a strutture allocate e inizializzate.
Effetti collaterali	Alloca memoria (malloc x3); in caso di fallimento termina il programma (exit).
Output / Ritorna	void.

libera_risorse

Sintassi	<code>void libera_risorse(TabellaHashStudenti* t, TurnoAula* aula, CodaAttesa* coda);</code>
Semantica	Rilascia tutta la memoria: nodi dei bucket, nodi della coda e le tre struct principali. Azzeri i puntatori di testa. Da chiamare una sola volta in chiusura.
Parametri (input)	<code>TabellaHashStudenti* t</code> — Tabella (NULL ammesso). <code>TurnoAula* aula</code> — Aula (NULL ammesso). <code>CodaAttesa* coda</code> — Coda (NULL ammesso).
Pre-condizione	Nessuna.
Post-condizione	Tutta la memoria delle strutture passate e' liberata.
Effetti collaterali	Libera memoria (free); stampa su stdout.
Output / Ritorna	void.

visualizza_studenti_per_stato

Sintassi	<code>void visualizza_studenti_per_stato(TabellaHashStudenti* anagrafica, TurnoAula* aula, CodaAttesa* coda);</code>
Semantica	Stampa gli studenti suddivisi in PRESENTI (OCCUPATO), PRENOTATI (PRENOTATO) e IN CODA, con matricola, nome e corso (placeholder se assenti).
Parametri (input)	<code>TabellaHashStudenti* anagrafica</code> — Anagrafica (non NULL). <code>TurnoAula* aula</code> — Stato aula (non NULL). <code>CodaAttesa* coda</code> — Coda (non NULL).
Pre-condizione	Sistema inizializzato.
Post-condizione	Nessuna modifica alle strutture.
Effetti collaterali	Stampa su stdout.
Output / Ritorna	void.

visualizza_situazione_corrente

Sintassi	<code>void visualizza_situazione_corrente(TurnoAula* aula, CodaAttesa* coda);</code>
Semantica	Vista compatta dello stato dell'aula (per posto e matricola) e della coda.

Parametri (input)	TurnoAula* aula — Stato aula (non NULL). CodaAttesa* coda — Coda (non NULL).
Pre-condizione	Sistema inizializzato.
Post-condizione	Nessuna modifica alle strutture.
Effetti collaterali	Stampa su stdout.
Output / Ritorna	void.

genera_report_aula

Sintassi	<code>void genera_report_aula(TurnoAula* aula, CodaAttesa* coda);</code>
Semantica	Stampa il report: contatori del turno, snapshot istantaneo, occupazione per fascia, barra di saturazione e storico letto da file.
Parametri (input)	TurnoAula* aula — Stato aula (non NULL). CodaAttesa* coda — Coda (non NULL).
Pre-condizione	Sistema inizializzato.
Post-condizione	Nessuna modifica alle strutture.
Effetti collaterali	Legge storico_accessi.txt; stampa su stdout.
Output / Ritorna	void.

salva_storico_accesso

Sintassi	<code>void salva_storico_accesso(TabellaHashStudenti* anagrafica, char* matricola, char* operazione, OrarioVirtuale ora);</code>
Semantica	Aggiunge una riga a storico_accessi.txt (append) con timestamp, dati anagrafici (o N/D) e operazione. Se anagrafica e' NULL salta il lookup.
Parametri (input)	TabellaHashStudenti* anagrafica — Anagrafica; NULL ammesso. char* matricola — Matricola dell'evento. char* operazione — Descrizione dell'operazione. OrarioVirtuale ora — Timestamp virtuale.
Pre-condizione	Il file deve essere scrivibile nella directory di lavoro.
Post-condizione	Una riga aggiunta in fondo al file (o errore su stderr).

Effetti collaterali	Scriva su file (I/O); può stampare su stderr.
Output / Ritorna	void.

visualizza_storico_accessi

Sintassi	<code>void visualizza_storico_accessi();</code>
Semantica	Legge storico_accessi.txt e lo stampa riga per riga; se assente lo segnala in modo non bloccante.
Parametri (input)	Nessuno.
Pre-condizione	Nessuna.
Post-condizione	Nessuna modifica al file.
Effetti collaterali	Legge da file (I/O); stampa su stdout.
Output / Ritorna	void.

mostra_menu

Sintassi	<code>void mostra_menu();</code>
Semantica	Stampa il menu principale; non legge input.
Parametri (input)	Nessuno.
Pre-condizione	Nessuna.
Post-condizione	Nessuna modifica.
Effetti collaterali	Stampa su stdout.
Output / Ritorna	void.

7. Programma principale (main.c)

main

Sintassi	<code>int main(void);</code>
Semantica	Punto di ingresso. Inizializza il sistema, poi in un loop do-while aggiorna l'orario virtuale, stampa lo stato, mostra il menu, legge la scelta e instrada verso l'operazione. Gli input numerici non validi non terminano il programma. Termina su scelta 0, liberando la memoria.
Parametri (input)	Nessuno.
Pre-condizione	Nessuna.
Post-condizione	Alla chiusura tutte le risorse sono liberate.
Effetti collaterali	Alloca/libera memoria; legge da stdin; scrive su stdout; scrive su storico_accessi.txt tramite le operazioni invocate.
Output / Ritorna	0 in caso di chiusura normale.

7.1 Variabili locali di main

Variabile	Significato
<code>TabellaHashStudenti* anagrafica</code>	Puntatore (opaco) all'anagrafica; allocato da <code>inizializza_sistema_dinamico</code> .
<code>TurnoAula* aula</code>	Puntatore (opaco) allo stato dell'aula.
<code>CodaAttesa* coda</code>	Puntatore (opaco) alla coda d'attesa.
<code>OrarioVirtuale ora_attuale</code>	Orario virtuale corrente; parte da 09:00:00.
<code>time_t ultimo_controllo</code>	Timestamp reale dell'ultimo aggiornamento dell'orario.
<code>int scelta</code>	Voce di menu scelta; controlla anche la condizione del loop.
<code>char mat[20]</code>	Buffer per la matricola immessa.
<code>char nome[60], corso[60]</code>	Buffer per nome e corso (registrazione).
<code>int scelta_f</code>	Indice numerico della fascia (0/1/2) nel case Prenotazione.
<code>FasciaOraria f_scelta</code>	Fascia derivata da <code>scelta_f</code> .
<code>char risp</code>	Risposta s/n alla registrazione al volo (case 2 e 3).
<code>char nome_preno[60], corso_preno[60]</code>	Buffer per la registrazione al volo (Prenotazione).

Variabile	Significato
<code>char nome_nuovo[60], corso_nuovo[60]</code>	Buffer per la registrazione al volo (Check-in).

8. Casi di Test (test.c)

La suite verifica i requisiti funzionali della traccia. Ogni test e' indipendente grazie a `reset_aula()`; le asserzioni usano la macro `ASSERT`, che stampa `[PASS]/[FAIL]` senza interrompere l'esecuzione.

Elemento	Significato
<code>ASSERT(cond, msg)</code>	Macro che stampa <code>[PASS]</code> se <code>cond</code> e' vera, <code>[FAIL]</code> altrimenti. Usa <code>do{...}while(0)</code> . Non interrompe la suite.

reset_aula

Sintassi	<code>static void reset_aula(TurnoAula* aula, CodaAttesa* coda);</code>
Semantica	Riporta l'aula a stato pulito tra i test, delegando a <code>cambio_fascia_automatica(aula, coda, MATTINA)</code> . Non tocca l'anagrafica.
Parametri (input)	<code>TurnoAula* aula</code> — Stato aula (non NULL). <code>CodaAttesa* coda</code> — Coda (non NULL).
Pre-condizione	Sistema inizializzato.
Post-condizione	Aula vuota, coda pulita dei nodi MATTINA, contatori a 0, fascia MATTINA.
Effetti collaterali	Libera nodi della coda; scrive su storico; stampa su stdout (tramite <code>cambio_fascia_automatica</code>).
Output / Ritorna	<code>void</code> .

test_registrazione

Sintassi	<code>static void test_registrazione(TabellaHashStudenti* anagrafica);</code>
Semantica	Verifica registrazione di 3 studenti, rifiuto del duplicato (nome preservato) e ricerca di matricola inesistente. Popola l'anagrafica per i test successivi.
Parametri (input)	<code>TabellaHashStudenti* anagrafica</code> — Anagrafica inizializzata (non NULL).
Pre-condizione	Sistema inizializzato.
Post-condizione	Anagrafica popolata con T001/T002/T003; risultati stampati.
Effetti collaterali	Alloca memoria; scrive su storico; stampa su stdout.

Output / Ritorna	void.
-------------------------	-------

test_prenotazione

Sintassi	<code>static void test_prenotazione(TabellaHashStudenti* anagrafica, TurnoAula* aula, CodaAttesa* coda);</code>
Semantica	Verifica prenotazione valida (+1 posto), rifiuto di matricola inesistente e di duplicato (contatore invariato).
Parametri (input)	<code>TabellaHashStudenti* anagrafica</code> — Non NULL, con T001 registrato. <code>TurnoAula* aula</code> — Non NULL. <code>CodaAttesa* coda</code> — Non NULL.
Pre-condizione	Sistema inizializzato; T001 registrato.
Post-condizione	Risultati stampati; reset a fine test.
Effetti collaterali	Modifica aula/coda; scrive su storico; stampa su stdout.
Output / Ritorna	void.

test_disponibilita

Sintassi	<code>static void test_disponibilita(TabellaHashStudenti* anagrafica, TurnoAula* aula, CodaAttesa* coda);</code>
Semantica	Verifica che i posti liberi calino di 1 dopo una prenotazione e tornino al valore iniziale dopo l'annullamento.
Parametri (input)	<code>anagrafica/aula/coda</code> — Strutture inizializzate (non NULL).
Pre-condizione	Sistema inizializzato.
Post-condizione	Risultati stampati; reset a fine test.
Effetti collaterali	Modifica aula/coda; scrive su storico; stampa su stdout.
Output / Ritorna	void.

test_checkin_checkout

Sintassi	<code>static void test_checkin_checkout(TabellaHashStudenti* anagrafica, TurnoAula* aula, CodaAttesa* coda);</code>
-----------------	---

Semantica	Verifica il ciclo PRENOTATO -> OCCUPATO (check-in) -> LIBERO (check-out con coda vuota).
Parametri (input)	anagrafica/aula/coda — Strutture inizializzate (non NULL).
Pre-condizione	Sistema inizializzato.
Post-condizione	Risultati stampati; reset a fine test.
Effetti collaterali	Modifica aula/coda; scrive su storico; stampa su stdout.
Output / Ritorna	void.

test_ingresso_senza_prenotazione

Sintassi	<pre>static void test_ingresso_senza_prenotazione(TabellaHashStudenti* anagrafica, TurnoAula* aula, CodaAttesa* coda);</pre>
Semantica	Caso A: coda vuota + posti liberi -> ingresso diretto. Caso B: coda non vuota -> il nuovo arrivato va in coda (priorita' FIFO).
Parametri (input)	anagrafica/aula/coda — Strutture inizializzate (non NULL).
Pre-condizione	Sistema inizializzato.
Post-condizione	Risultati stampati; reset a fine test.
Effetti collaterali	Modifica aula/coda (malloc/free); scrive su storico; stampa su stdout.
Output / Ritorna	void.

test_lista_attesa

Sintassi	<pre>static void test_lista_attesa(TabellaHashStudenti* anagrafica, TurnoAula* aula, CodaAttesa* coda);</pre>
Semantica	Scenario 1: coda riempita con MAX_POSTI nodi -> il check-in successivo va in coda. Scenario 2: subentro reale -> l'uscita di T001 promuove T002.
Parametri (input)	anagrafica/aula/coda — Strutture inizializzate (non NULL).
Pre-condizione	Sistema inizializzato.
Post-condizione	Risultati stampati; reset tra gli scenari e a fine test.

Effetti collaterali	Modifica aula/coda (malloc/free); scrive su storico; stampa su stdout.
Output / Ritorna	void.

test annullamento

Sintassi	<code>static void test annullamento(TabellaHashStudenti* anagrafica, TurnoAula* aula, CodaAttesa* coda);</code>
Semantica	Tre scenari: A) nessuno in coda -> posto LIBERO; B) coda stessa fascia -> subentro; C) coda fascia diversa -> posto LIBERO e coda invariata.
Parametri (input)	<code>anagrafica/aula/coda</code> — Strutture inizializzate (non NULL).
Pre-condizione	Sistema inizializzato.
Post-condizione	Risultati stampati; reset tra gli scenari e a fine test.
Effetti collaterali	Modifica aula/coda (malloc/free); scrive su storico; stampa su stdout.
Output / Ritorna	void.

test storico_report

Sintassi	<code>static void test storico_report(TabellaHashStudenti* anagrafica, TurnoAula* aula, CodaAttesa* coda);</code>
Semantica	Esegue una storia tipica (prenotazione+checkin+checkout), verifica l'esistenza del file storico e chiama <code>genera_report_aula</code> come smoke test.
Parametri (input)	<code>anagrafica/aula/coda</code> — Strutture inizializzate (non NULL).
Pre-condizione	Sistema inizializzato.
Post-condizione	File storico verificato; report stampato; reset a fine test.
Effetti collaterali	Legge/scrive su file; modifica aula/coda; stampa su stdout.
Output / Ritorna	void.

main (test)

Sintassi	<code>int main(void);</code>
-----------------	------------------------------

Semantica	Inizializza il sistema, esegue tutti i test (registrazione per prima) e libera la memoria. Non si interrompe in caso di [FAIL].
Parametri (input)	Nessuno.
Pre-condizione	Nessuna.
Post-condizione	Tutti i test eseguiti; memoria liberata.
Effetti collaterali	Alloca/libera memoria; scrive su storico; stampa su stdout.
Output / Ritorna	0 sempre.

8.1 Variabili locali ricorrenti nei test

Variabile	Significato
<code>OrarioVirtuale ora</code>	Orario fittizio (di norma 09:30:00) usato come timestamp.
<code>int occupati_prima</code>	Posti occupati prima dell'operazione (baseline).
<code>int liberi_prima / liberi_dopo</code>	Posti liberi (MAX_POSTI - occupati) prima/dopo.
<code>int dim_prima</code>	Dimensione della coda prima di un'operazione (baseline).
<code>Studente* trovato</code>	Risultato di <code>cerca_studente</code> verificato nelle asserzioni.
<code>FILE* fp</code>	Handle al file storico per verificarne l'esistenza.
<code>int i</code>	Indice dei cicli di riempimento (nodi FILL).